McGill

DECEMBER 2022
FINAL EXAMINATION

Applied Machine Learning

COMP 551 001

December 14 9:00 am – 12:00 pm

EXAMINER: Yue Li ASSOC. EXAMINER: Reihaneh Rabbany

STUDENT NAME:		**McGILL ID:**									

EXAM:	CLOSED BOOK ☐ OPEN BOOK ☒
	PRINTED ON BOTH SIDES OF THE PAGE ☐ SINGLE-SIDED ☒
	MULTIPLE CHOICE ANSWER SHEETS: YES ☐ NO ☒ NOTE: The Examination Security Monitor Program detects pairs of students with unusually similar answer patterns on multiple-choice exams. Data generated by this program can be used as admissible evidence, either to initiate or corroborate an investigation or a charge of cheating under Section 17 of the Code of Student Conduct and Disciplinary Procedures. ANSWER BOOKLET REQUIRED: YES ☐ NO ☒ EXTRA BOOKLETS PERMITTED: YES ☐ NO ☒ ANSWER ON EXAM: YES ☒ NO ☐
	SHOULD THE EXAM BE: RETURNED ☒ KEPT BY STUDENT ☐
CRIB SHEETS:	PERMITTED ☒ NOT PERMITTED ☐
DICTIONARIES:	TRANSLATION ☒ REGULAR ☒ NOT PERMITTED ☐
CALCULATORS:	NOT PERMITTED ☐ PERMITTED (Non-Programmable) ☒ PERMITTED (Programmable) ☐
ANY SPECIAL INSTRUCTIONS:	

- You have 180 minutes to write the exam.
- Hard-copy notes, books, and printed slides are allowed but electronic devices are NOT allowed.
- This exam contains 24 questions on 18 pages for total of 100 + 2 bonus points.
- 10 multiple-choice questions: circle only ONE correct answer per question.
- 4 multiple-select questions, circle ALL correct answers per question. Scoring: Right minus Wrong.
- 9 short-answer questions (including a bonus): write your answer directly below each question.
- Advice: Try not to spend too much time searching answers through your notes as it will slow you down and you will not have enough time to complete this exam in 3 hours. Good luck!

1 Multiple-choice questions (30 points)

1. (3 points) Of the following supervised learning methods, which one *always* requires the presence of *the entire* training data in order to make prediction on any test data?

   A. K nearest neighbour

   B. Decision tree

   C. Logistic regression

   D. MLP

   E. Support vector machine

2. (3 points) In the context of bias-variance trade-off, which of the following actions will *most likely* decrease *both* the model variance and the bias?

   A. Increase tree-depth in the regression tree.

   B. Increase regularization in ridge regression.

   C. Increase K in K-nearest neighbours.

   D. Increase the number of hidden units in each layer of the MLP.

   E. Increase the number of trees in Random Forest.

3. (3 points) Suppose we have a function $\hat{y} = f(\mathbf{x}; \theta)$, where $\mathbf{x} \in \mathcal{R}^D$, $\hat{y} \in \mathcal{R}$ (i.e., $\hat{y}$ is a real number), and θ are the learnable model parameters. To train this model for a *binary classification* task, choose the correct *loss function* below.

   A. $\mathcal{L} = (\hat{y} - y)^2$

   B. $\mathcal{L} = -y \log \hat{y} - (1 - y) \log \hat{y}$

   C. $\mathcal{L} = y \log \sigma(\hat{y}) + (1 - y) \log(1 - \sigma(\hat{y}))$, where $\sigma(\hat{y}) = (1 + \exp(-\hat{y}))^{-1}$

   D. $\mathcal{L} = -y \log \sigma(\hat{y}) - (1 - y) \log(1 - \sigma(\hat{y}))$**, where** $\sigma(\hat{y}) = (1 + \exp(-\hat{y}))^{-1}$

4. (3 points) After training a binary classifier that can produce probability for the positive class, what threshold (if any) guarantees to produce 100% *True Positive Rate* on the test data? Note we set the predicted class to 1 if the model predicted probability is *greater* than the threshold.

   A. -1

   B. 0.5

   C. 1

   D. No such threshold exists

Solution: For TPR=TP/(TP+FN), we can have TPR equal to 1 when the threshold is -1. That is, every data point is predicted to be positive and have zero false negative.

5. (3 points) Suppose we have a training dataset with 1000 features but only 50 training examples. What's the main concern in choosing a deep neural network to fit such data?

   A. Overfitting
   B. Underfitting
   C. Model convergence
   D. Hyperparameter tuning
   E. Vanishing gradient
   F. Gradient explosion

6. (3 points) What technique is the key that allows us to efficiently train a neural network on millions of training examples?

   A. Batch normalization
   B. Bayesian optimization
   C. Regularization
   D. ReLU activation
   E. Stochastic gradient descent

7. (3 points) How many parameters in total for a 2-layer MLP of 9 input units, 5 hidden units in each layer and 3 output units? Assuming there is a bias term for each hidden unit and output unit.

   A. 85
   B. 94
   C. 98
   D. 135

Solution: Correct: $(9+1)\times 5+(5+1)\times 5+(5+1)\times 3=50+30+18=98$
Incorrect: $9\times 5+5\times 5+5\times 3=45+25+15=85$

8. (3 points) Which model has *the smallest number* of parameters in training to predict whether a 28×28 MNIST image is a written digit "2" (i.e., a binary classifier; assuming no bias)?

   A. A ConvNet with a single convolutional filter of size 28×28 (no padding or stride) directly followed by a sigmoid transformation to produce output probability $\hat{y}$.
   B. A ConvNet with a single convolutional filter of size 1×1 (no padding or stride) followed by flattening and then an output layer to produce output probability $\hat{y}$.
   C. A 1-layer MLP with 1 hidden unit in its hidden layer followed by a sigmoid transformation to produce output probability $\hat{y}$.

Solution:

- The ConvNet with a 28×28 filter has parameters size equal to 28×28

- The ConvNet with a 1×1 filter need to have a dense layer of $(28 \times 28) \times 1$ and therefore has parameters $1 + 28 \times 28$
- The 1-layer MLP has $28 \times 28 = 784$ input-to-hidden and then a 1 parameter to go from hidden to output and therefore also has $28 \times 28 + 1$ parameters

9. (3 points) Suppose we are designing a recurrent neural network (RNN) for diagnosing patients after 5 days during their 10-day hospital stay. That is, the RNN model takes a sequence of input $\mathbf{x}_1, \ldots, \mathbf{x}_5$ and makes prediction for $\mathbf{y}$ at the end of the fifth day. Which of the following will *not* increase the number of parameters in such RNN?

A. Increasing the number of hidden units

B. Instead of 5 days, we decide to let the RNN train on 7-day of the time-series data to more accurately diagnose patients

C. Increasing the number of hidden layers

D. Adding more static features such as patient age, sex, and pre-existing medical conditions.

E. All of the above will increase the number of parameters for the RNN

Solution: Because the weights are shared from time t to $t+1$, increasing the length of the input sequence will not increase the RNN parameters

10. (3 points) Suppose you are training a LASSO model with the penalty tuning parameter λ set to 1 on a training data with $N = 10$ training examples. Each input feature is *standardized* to have mean equal to 0 and standard deviation equal to 1. After iteration $t-1$, the value for the estimated coefficient $\hat{w}_d^{\{t-1\}}$ for feature d is 0.5. *Without the ℓ_1 penalty*, the partial derivative of the Sum of Squared Error (SSE) w.r.t. w_d is 0.1 (while fixing all other coefficients $\hat{\mathbf{w}}_{\setminus d}^{\{t-1\}}$). That is, $-\mathbf{x}_d^\top(\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) + w_d||\mathbf{x}_d||_2^2 = 0.1$ where $\hat{\mathbf{y}}_{\setminus d} = \mathbf{X}\hat{\mathbf{w}}_{\setminus d}^{\{t\}}$. Recall in Module 4.5 that the LASSO update rule is:

$$\hat{w}_d = \begin{cases} \frac{\mathbf{x}_d^\top(\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) + \lambda}{||\mathbf{x}_d||_2^2} & \mathbf{x}_d^\top(\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) < -\lambda \\ 0 & \mathbf{x}_d^\top(\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) \in [-\lambda, \lambda] \\ \frac{\mathbf{x}_d^\top(\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) - \lambda}{||\mathbf{x}_d||_2^2} & \mathbf{x}_d^\top(\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) > \lambda \end{cases}$$

What is the new LASSO estimate for $\hat{w}_d^{\{t\}}$ at iteration t?

A. -0.49

B. -0.39

C. 0

D. 0.39

E. 0.49

Solution: To apply the update LASSO rule, we need to compute $\mathbf{x}_d^\top(\mathbf{y} - \hat{\mathbf{y}}_{\setminus d})$. Since the data are standardized, we know that $||x_d||_2^2 = N = 10$. Therefore, $\mathbf{x}_d^\top(\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) = 4.9 > \lambda = 1$. Applying the third scenario in the update rule, $\hat{w}_d^{\{t\}} = (4.9 - 1)/10 = 0.39$.

2 Multiple-select questions (20 points)

11. (4 points) What regression model(s) below is or are equivalent to a model, which simply takes the average of the response values in the training data as the predicted value for all test data points?
    A. Regression tree with only the root node
    B. K-nearest neighbours with K set to be 1
    C. K-nearest neighbours with K set to be the number of training examples
    D. Linear regression that only fits the bias term, i.e., $\hat{y}^{(n)} = w_0$.

12. (4 points) Circle ALL method(s) that can be properly trained by gradient descent.
    A. Decision tree
    B. Linear regression without regularization
    C. Linear regression with ℓ_1-regularization
    D. Logistic regression with ℓ_2-regularization

13. (4 points) Choose ALL technique(s) that can reduce the number of parameters of a convolutional neural network (considering everything else fixed).
    A. Decrease the number of convolutional filters
    B. Increase stride size at every convolutional layer
    C. Increase average pooling size at each convolutional layer
    D. Decouple the filter of dimension $H \times W \times C \times D$ for height, width, channel, and depth (output channel) by using a 2D filter W of dimension $H \times W$ and another 2D filter of dimension $C \times D$ to perform depthwise separable convolution.

14. (4 points) We are training a recurrent neural network (RNN) on IMDB movie review data to predict good and bad movies by treating each review as sequential text data. Choose ALL plausible cause(s) of vanishing gradient.
    A. Too many hidden layers in the RNN.
    B. Too many hidden units at each hidden layer in the RNN.
    C. Some movie reviews are too long.
    D. Sigmoid function is used to activate each hidden unit.

15. (4 points) Connect each problem on the left to the most suitable deep learning method on the right.

• Predicting spams using hand-crafted keyword features	• RNN for Seq2Seq unaligned case
• Translating English sentences to French sentences	• MLP with sigmoid output
• Predicting cancer stages from tumor images	• RNN for Seq2Vec
• Name entity recognition in sentences	• RNN for Seq2Seq aligned case
• Predicting content categories from unstructured text reports	• Convolutional neural network + MLP with softmax outputs

Solution:

- Predict spams using handcrafted keyword features, MLP with sigmoid output
- Translating English sentence to French sentence, RNN for Seq2Seq unaligned case
- Predicting cancer stages from tumor images, Convolutional neural network + MLP with softmax outputs
- Name entity recognition in sentences, RNN for Seq2Seq aligned case
- Predicting content categories from the unstructured text reports, RNN for Seq2Vec

3 Short answer questions (50 points)

16. (2 points) When performing Stochastic Gradient Descent (SGD) using a simplified version of Adams update as shown below,

    - (1 points) What happens if we increase β_1 in the update equation below?
    - (1 points) What happens if we increase β_2 in the update equation below?

$$\mathbf{M}^{\{t\}} \leftarrow \beta_1 \mathbf{M}^{\{t-1\}} + (1-\beta_1)\nabla J(\mathbf{w}^{\{t-1\}})$$
$$\mathbf{S}^{\{t\}} \leftarrow \beta_2 \mathbf{S}^{\{t-1\}} + (1-\beta_2)\left(\nabla J(\mathbf{w}^{\{t-1\}})\right)^2$$
$$\mathbf{w}^{\{t\}} \leftarrow \mathbf{w}_d^{\{t-1\}} - \frac{\alpha}{\sqrt{\mathbf{S}^{\{t\}} + \epsilon}}\mathbf{M}^{\{t\}}$$

Solution: Increasing β_1 means increasing momentum [0.5 pts] and smoother path with less oscillation in SGD [0.5 pts]. Increasing β_2 means preserving more of the earlier changes in the learning step [1 pts].

17. (2 points) In Group LASSO, we have the L2-norm to penalize each group of coefficients:

$$J(\mathbf{w}) = \frac{1}{2}||\mathbf{y} - \mathbf{X}\mathbf{w}||_2^2 + \lambda \sum_{g=1}^{G} ||\mathbf{w}_g||_2, \quad \text{where} \quad ||\mathbf{w}_g||_2 = \sqrt{\sum_{d \in \mathcal{S}_g} w_d^2}$$

What difference does it make if we used *squared* L2-norm $\sum_{g=1}^{G} ||\mathbf{w}_g||_2^2$ instead of L2-norm as the weight penalty in the Group LASSO?

Solution: The Group LASSO reduces to ridge regression when squared L2-norm is used as the weight penalty because $\sum_{g=1}^{G} ||\mathbf{w}_g||_2^2 = \sum_{g=1}^{G} \sum_{d \in \mathcal{S}_g} w_d^2 = \sum_{d=1}^{D} w_d^2$

18. (2 points) When training a multi-class prediction model, some of the class scores computed by $a_c = \mathbf{x}_c \mathbf{w}_{.,c}$ for $c \in \{1, \ldots, C\}$ can be a large positive number (e.g., $a_c = 1000$), which can result in infinity value on the computer when applying the softmax transformation to get predicted class probabilities (i.e., $\exp(a_c)/\sum_{c'} \exp(a'_c)$). How to resolve this issue and still obtain the same class probabilities as if our computers could handle the exponential of any large number?

Solution: Before applying softmax, we can subtract each class score by the sum or max of the class scores. This is the same as dividing both the numerator and the denominator by a constant such that the class probabilities will not change:

$$\frac{\exp(a_c - \sum_k a_k)}{\sum_{c'} \exp(a_{c'} - \sum_k a_k)} = \frac{\exp(a_c)/\exp(\sum_k a_k)}{\sum_{c'} \exp(a_{c'})/\exp(\sum_k a_k)} = \frac{\exp(a_c)}{\sum_{c'} \exp(a_{c'})}$$

[YL: If they said they divide top and bottom with a large number like 1000, give them one point.]

19. (8 points) Describe an MLP that is equivalent to a multiple linear regression model $\hat{y} = \sum_{d=1}^{100} \phi_d(x) w_d$ that operates on 100 non-linear basis features computed from 100 Gaussian basis functions, each of which transforms the same 1D feature x by:

$$\phi_d(x) = \exp(-\frac{x^2}{2\sigma_d^2}), \quad \text{where} \quad \sigma_d = \sqrt{d/2}$$

Ignore the bias in this question.

- (2 pts) What's the architecture of such MLP?
- (2 pts) What are the input-to-hidden weights in the MLP?
- (2 pts) What's the activation function for each hidden unit?
- (2 pts) What are the hidden-to-output weights?

Solution:

- (2 pts) This is equivalent to a 1-layer MLP with 100 hidden units and an output unit.
- (2 pts) The weight from the 1D feature x to each hidden unit is $v_d = \frac{1}{\sqrt{d}}$ because

$$\frac{x^2}{2\sigma_d^2} = \frac{x^2}{2\frac{d}{2}} = \frac{x^2}{d} = \left(\frac{x}{\sqrt{d}}\right)^2 \triangleq (v_d x)^2 \triangleq a_d^2$$

- (2 pts) Given $a_d = v_d x$, the activation function for each hidden unit is:

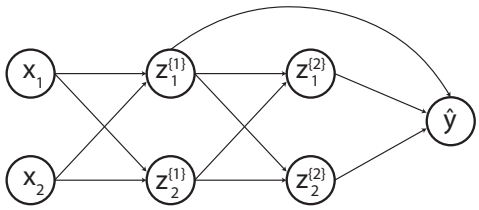
$$f(v_d x) = \exp(-a_d^2)$$

- (2 pts) The hidden-to-output weights are the same as the 200 linear regression coefficients $\{w_d\}_{d=1}^{D}$.

We can accept any other equivalent answer.

20. (10 points) The computational graph below illustrates the forward pass of a small neural network:

$$
\begin{aligned}
z_i^{\{1\}} &= \sigma(x_1 w_{1,i}^{\{1\}} + x_2 w_{2,i}^{\{1\}}) \quad \forall i \in \{1,2\} \\
z_j^{\{2\}} &= \sigma(z_1^{\{1\}} w_{1,j}^{\{2\}} + z_2^{\{1\}} w_{2,j}^{\{2\}}) \quad \forall j \in \{1,2\} \\
\hat{y} &= z_1^{\{2\}} w_1^{\{o\}} + z_2^{\{2\}} w_2^{\{o\}} + z_1^{\{1\}} w_3^{\{o\}} \\
\mathcal{L} &= (y - \hat{y})^2
\end{aligned}
$$

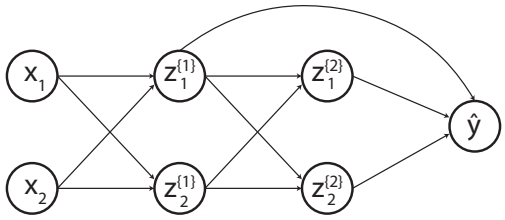


where $\sigma(x)$ is the sigmoid function. Assume we are performing automatic differentiation.

- (5 pts) Clearly write down the steps needed to obtain the partial derivative of $\frac{\partial \mathcal{L}}{\partial z_1^{\{1\}}}$ in the *forward mode*

- (5 pts) Clearly write down the steps needed to obtain $\frac{\partial \mathcal{L}}{\partial z_1^{\{1\}}}$ in the *backward mode*

Solution: [YL: For this question, it's ok if they write the expressions and recycle them in the later gradient chain rule as long as they expressed all of them in analytical forms.]



$$
\begin{aligned}
z_i^{\{1\}} &= \sigma(x_1 w_{1,i}^{\{1\}} + x_2 w_{2,i}^{\{1\}}) \quad \forall i \in \{1,2\} \\
z_j^{\{2\}} &= \sigma(z_1^{\{1\}} w_{1,j}^{\{2\}} + z_2^{\{1\}} w_{2,j}^{\{2\}}) \quad \forall j \in \{1,2\} \\
\hat{y} &= z_1^{\{2\}} w_1^{\{o\}} + z_2^{\{2\}} w_2^{\{o\}} + z_1^{\{1\}} w_3^{\{o\}} \\
\mathcal{L} &= (y - \hat{y})^2
\end{aligned}
$$

- Forward mode (from left to right):

$$\begin{aligned}
\frac{\partial z_1^{\{1\}}}{\partial z_1^{\{1\}}} &= 1 \\
\frac{\partial z_1^{\{2\}}}{\partial z_1^{\{1\}}} &= z_1^{\{2\}}(1 - z_1^{\{2\}}) w_{1,1}^{\{2\}} \\
\frac{\partial z_2^{\{2\}}}{\partial z_1^{\{1\}}} &= z_2^{\{2\}}(1 - z_2^{\{2\}}) w_{1,2}^{\{2\}} \\
\frac{\hat{y}}{\partial z_1^{\{1\}}} &= \frac{\partial z_1^{\{2\}}}{\partial z_1^{\{1\}}} w_1^{\{o\}} + \frac{\partial z_1^{\{2\}}}{\partial z_1^{\{1\}}} w_2^{\{o\}} + \frac{\partial z_1^{\{1\}}}{\partial z_1^{\{1\}}} w_3^{\{o\}} \\
\frac{\partial \mathcal{L}}{\partial z_1^{\{1\}}} &= \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\hat{y}}{\partial z_1^{\{1\}}} = -2(y - \hat{y}) \frac{\hat{y}}{\partial z_1^{\{1\}}} \\
&= -2(y - \hat{y}) \left(z_1^{\{2\}}(1 - z_1^{\{2\}}) w_{1,1}^{\{2\}} w_1^{\{o\}} + z_2^{\{2\}}(1 - z_2^{\{2\}}) w_{2,2}^{\{2\}} w_2^{\{o\}} + w_3^{\{o\}} \right)
\end{aligned}$$

- Backward mode (from right to left):

$$\begin{aligned}
\frac{\partial \mathcal{L}}{\partial \hat{y}} &= -2(y - \hat{y}) \\
\frac{\partial \mathcal{L}}{\partial z_1^{\{2\}}} &= \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_1^{\{2\}}} = -2(y - \hat{y}) w_1^{\{o\}} \\
\frac{\partial \mathcal{L}}{\partial z_2^{\{2\}}} &= \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_2^{\{2\}}} = -2(y - \hat{y}) w_2^{\{o\}} \\
\frac{\partial \mathcal{L}}{\partial z_1^{\{1\}}} &= \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_1^{\{1\}}} + \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_1^{\{2\}}} \frac{\partial z_1^{\{2\}}}{\partial z_1^{\{1\}}} + \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_2^{\{2\}}} \frac{\partial z_2^{\{2\}}}{\partial z_1^{\{1\}}} \\
&= -2(y - \hat{y}) \left(z_1^{\{2\}}(1 - z_1^{\{2\}}) w_{1,1}^{\{2\}} w_1^{\{o\}} + z_2^{\{2\}}(1 - z_2^{\{2\}}) w_{2,2}^{\{2\}} w_2^{\{o\}} + w_3^{\{o\}} \right)
\end{aligned}$$

21. (10 points) Suppose we are designing a convolutional neural network to convolve 3×3 color images of 3 color channels. For each input color channel and output channel, our ConvNet has a unique filter with size of 2×2. The padding of size 1 is applied to each color channel of the input image. No stride or dilation is applied. ReLU activation is applied to the outputs of the linear convolution.

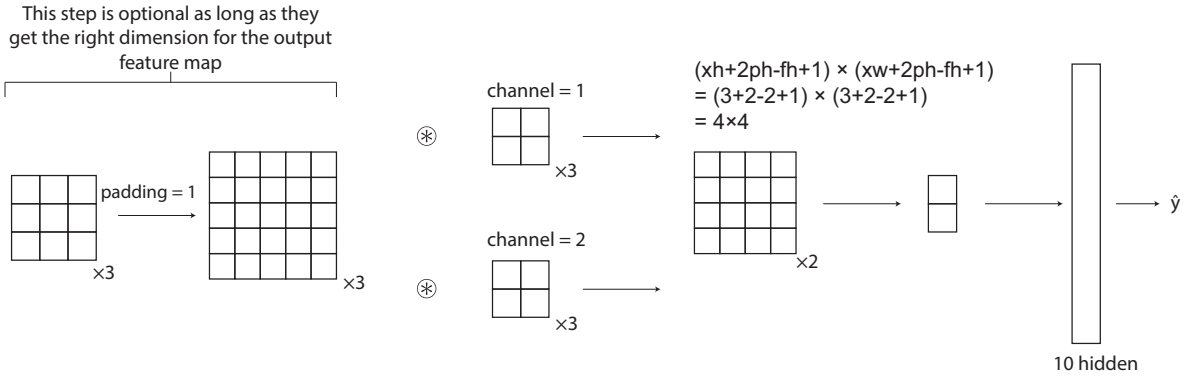
    The ConvNet has two output channels. A global max-pooling is applied to the convolved feature map.

    The results of the global pooling are then fed into a dense layer of 10 hidden units followed by a sigmoid output unit for prediction (e.g., whether the image contains a cat).

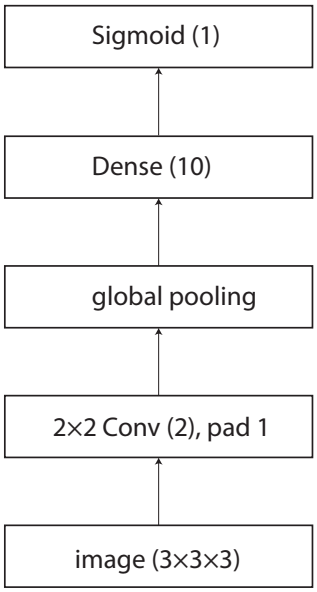
    - (5 pts) Based on the above description, draw the ConvNet architecture in detailed view.
    - (3 pts) Draw the same ConvNet in the compact view.
    - (2 pts) Assuming no bias term, how many learnable parameters are in this ConvNet?

Solution:

- Detailed view:



- Compact view:

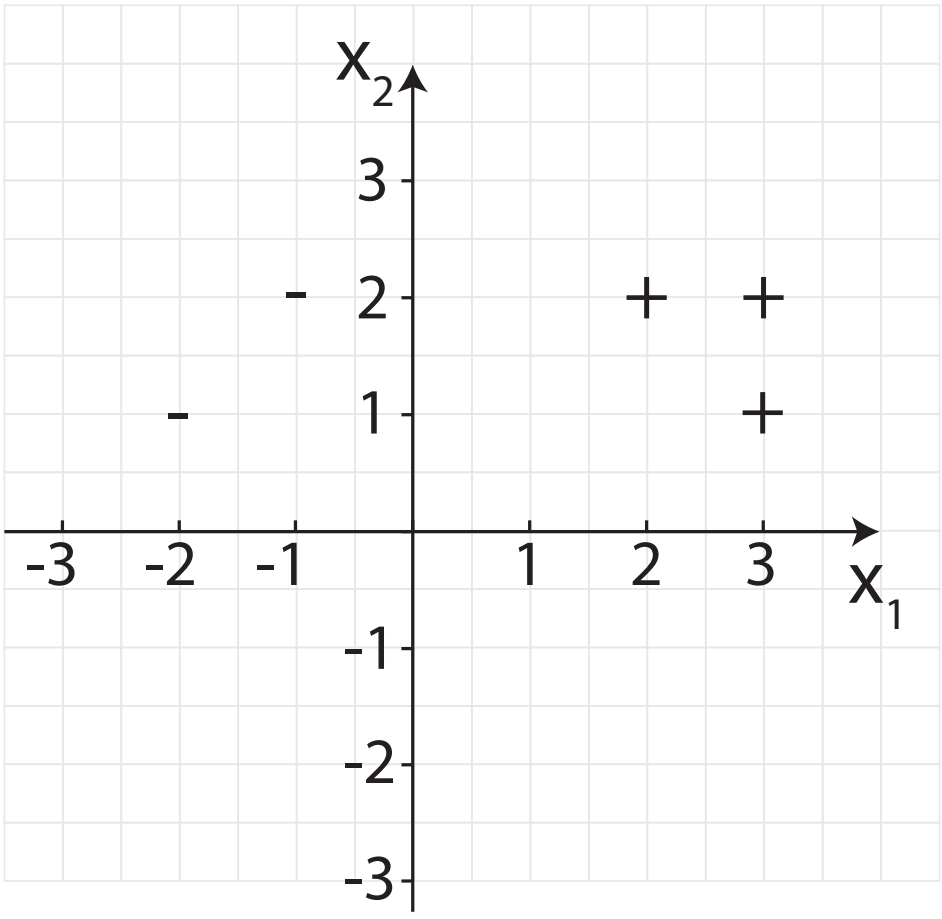


- # Parameters: $3 \times 2 \times 2 \times 2 + 2 \times 10 + 10 = 54$

22. (6 points) A Perceptron trained to separate 2D data points (x_1,x_2) has parameters set to be w_0 = -2 (bias), w_1 = 4 for x_1, and w_2 = 2 for x_2.

- (3 pts) Compute the decision boundary of the Perceptron and draw it directly on the plot below.
- (3 pts) Suppose the training examples are {(-2,1), -1}, {(-1,2), -1}, {(2,2), +1}, {(3,1), +1}, and {(3,2), +1} (in the format of $\{(x_1^{(n)}, x_2^{(n)}), \tilde{y}^{(n)}\}$) as shown on the plot below. What are the
  - support vectors (1 pt),
  - margins (1 pt),
  - and decision boundary (1 pt)

  determined by the hard-margin SVM classifier? Write down and draw your answers on the plot.



Solution:

- Its decision boundary is defined by all pairs of x_1 and x_2. We can solve the Perceptron equation $w_0 + w_1 x_1 + w_2 x_2 = 0$ for either x_1 or x_2:

$$x_1 = -\frac{w_0}{w_1} - \frac{w_2}{w_1} x_2 = 0.5 - 0.5 x_2$$

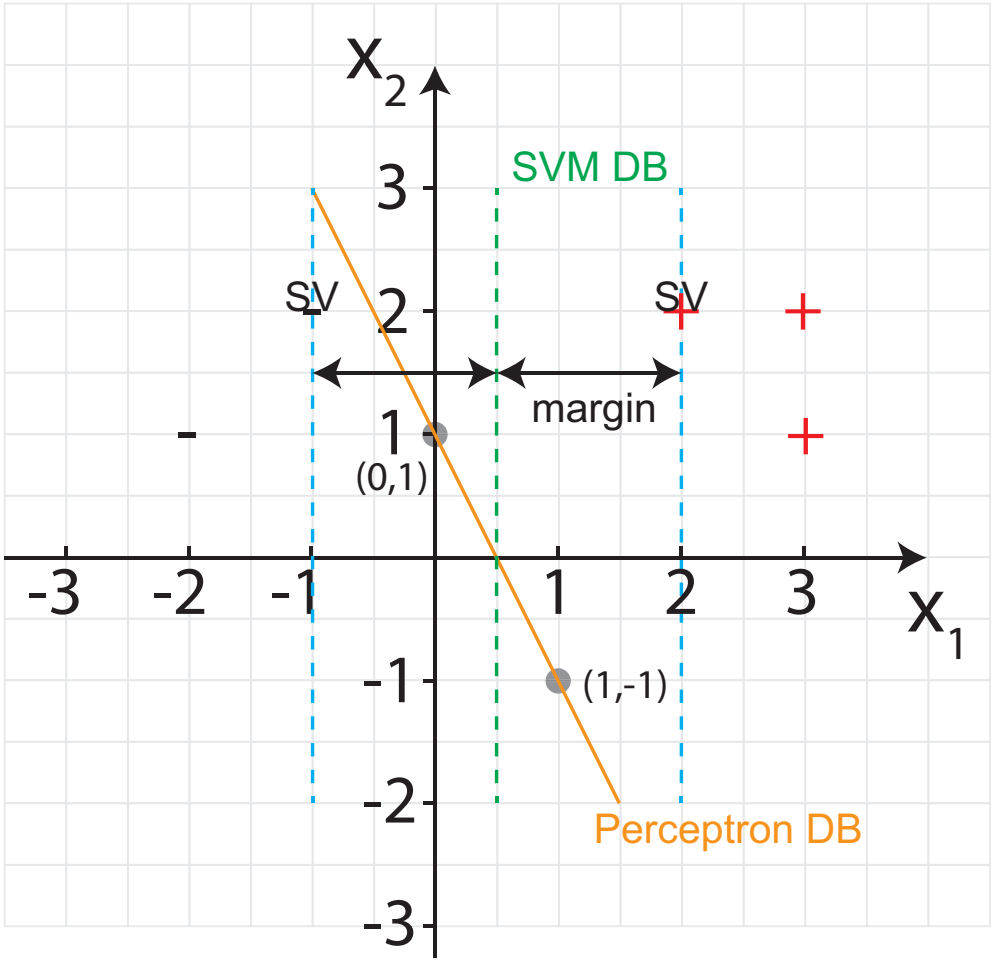
or

$$x_2 = -\frac{w_0}{w_2} - \frac{w_1}{w_2}x_1 = 1 - 2x_1$$

Given x_1 (or x_2), there is a unique point value for x_2 (or x_1), which defines the decision boundary of Perceptron.

- The SVM hard-margin chooses the training examples at (-1,2) and (2,2) as the support vectors to define its margins because they are the closest to the decision boundary that result in the maximum margin of 1.5.

  If they have their decision boundary and margin rotated to some degree, it is still correct as long as the margins pass through the same data points and it does not reduce the margin size.



23. (5 points) Describe the transformer encoder block. Highlight the key elements and computation needed to obtain the multi-headed attention (3 pts). Also, describe how we can go deeper by stacking transformer encoder blocks (2 pts). You may use equations to provide concise answer.

Solution: Let i denote the index of the attention head and H_ℓ be the number of attention heads at layer ℓ. When $\ell = 1$, $\mathbf{Z}^{\{1\}} = \mathbf{X}$. Word embedding from the hidden layer $\ell - 1$ to hidden layer ℓ are computed as follows:

$$\text{Query:} \quad \mathbf{Q}_i^{\{\ell\}} = \mathbf{Z}^{\{\ell-1\}}\mathbf{W}_i^{(q)}; \quad \text{Key:} \quad \mathbf{K}_i^{\{\ell\}} = \mathbf{Z}^{\{\ell-1\}}\mathbf{W}_i^{(k)}; \quad \text{Value:} \quad \mathbf{V}_i^{\{\ell\}} = \mathbf{Z}\mathbf{W}_i^{(v)}$$

$$\text{Self-attention:} \quad \mathbf{A}_i^{\{\ell\}} = \text{Attn}(\mathbf{Q}_i^{\{\ell\}}, \mathbf{K}_i^{\{\ell\}}); \quad \text{Word embedding from head } i\text{:} \quad \mathbf{Z}_i^{\{\ell\}} = \mathbf{A}_i^{\{1\}}\mathbf{V}_i^{\{1\}}$$

Concatenating the embedding from all heads:

$$\mathbf{Z}^{\{\ell\}} = [\mathbf{Z}_1^{\{\ell\}}, \ldots, \mathbf{Z}_{H_\ell}^{\{\ell\}}]$$

24. (5 points) Briefly describe with one sentence each the pre-training tasks of Word2vec skip-gram (1 pt), Embeddings from Language Models (ELMo) (1 pt), Generative Pre-trained Transformer (GPT) (1 pt), Bidirectional Encoder Representations from Transformers (BERT) (1 pt), and an example of a fine-tuning task after pre-training by any of the above models (1 pt).

Solution:

- Word2vec skip-gram predicts the flanking words based on the middle words via an MLP. nn
- ELMo pre-trains by predicting each word based on both the past and future sequence using a bi-directional LSTM.
- GPT pre-trains by predicting the next word a transformer decoder in an auto-regressive way using the past sequence.

- BERT predicts randomly masked words and/or next sentence using transformer-based self-attention.
- Sentiment prediction, NER, sentence pair entailment, neural machine translation, and any other reasonable example can be the answer here.

You may use this page as sketch.

You may use this page as sketch.